SOFTWARE ARCHITECTURES FOR ENTERPRISES · UNIVERSITÄT TRIER

ReflectAndMore

A field guide to Java meta-programming, or: how enterprise frameworks work, built by hand in eight stages

```
Candidate c = new Candidate(7);
// From here on, nothing about c is private any more.
```

Peter Sturm

Winter term 2026/27 · written with Claude

Contents

Introduction 4
- The small chill 4
- Who this is for 4
- Meet Candidate 5
- How the book and the code fit together 6
- The map 6
- A note on tone 7

Plain Java 8
- The program 8
- What the compiler decided 9
- Four things this program cannot do 9
- Where the names live 10

Run-time type information 12
- Where we stand 12
- Every object knows its class 12
- Class objects are ordinary values 13
- From a string to a type 14
- Walking the hierarchy 15
- Type tests and casts, without the syntax 15
- The compiler uses RTTI too 16
- Types that describe themselves 16
- Generics: erased from objects, kept in declarations 16
- Lambdas and anonymous classes have classes too 18

Reflection 20
- Where we stand 20
- Inspector: describing a class you have never seen 20
- Creating objects by name 23
- Calling methods by name 24
- Reading and writing fields, even final ones 24
- The wall 25
- Arrays 26
- Invoker: a convenience layer 26
- What does it cost? 28

Annotations 30
- Where we stand 30
- Defining an annotation and reading it back 30
- Meta-annotations: annotations all the way down 31
- Inspector lists annotations without knowing any 32
- Retention: who can see the annotation? 33
- Repeatable and inherited 33
- Annotations are interfaces; instances are proxies 34
- Framework one: declarative validation 35

Framework two: a test runner 36

Introduction

In which we promise to take the magic out of enterprise Java, and introduce the most boring class ever written.

The small chill

Every Java developer has had the moment. You write a class, you put `@Component` on top of it, you declare a field with `@Inject` or `@Autowired`, and you never write `new`. The object simply *is there* when you need it, with all its collaborators attached. You add `@Transactional` to a method and a database transaction wraps itself around every call. You add `@Test` and a method you never call is called for you. You return an object from a REST endpoint and it leaves the building as JSON.

It works, and it is wonderful, and somewhere in the back of your mind there is a small chill: *I have no idea how any of this happens.*

This book is about that chill. In eight stages it builds, from nothing but the JDK, every mechanism that makes a modern application framework tick: run-time type information, reflection, annotations, dynamic proxies, dynamic class loading, a dependency-injection container, and finally classes written as raw bytecode while the program runs. At the end you will have a tiny Spring, a tiny JUnit, a tiny Hibernate proxy and a tiny RMI in a few hundred lines, and you will know exactly why they work and what they cost. Frameworks are not magic. They are reflection, annotations, proxies and class loaders, arranged with taste.

Who this is for

You can program in Java. You know what an interface is, you have written a class with a constructor, and you have used a `List<String>`. You have probably used a framework and felt the chill. You have never, or hardly ever, called `getClass()` on purpose, and you have never defined your own annotation. That is the audience: people who are good at the language and have not yet looked under its floorboards.

Everything here is plain JDK 25. There are no third-party libraries, because a library would be another place for magic to hide. If a program in this book does something, you can open the source and see the whole mechanism.

Meet Candidate

Textbooks love to explain reflection with a `Customer` class that has an `Order` with `LineItems`. We will not do that. A business domain is a distraction: half of your attention goes to wondering whether the discount logic is realistic, and the mechanism being explained gets the other half. So our protagonist is deliberately, proudly boring. It is called `Candidate`, because it is the candidate for whatever we do next: inspection, interrogation, wiretapping, cloning.

```
public class Candidate {

    private static int created;        // counts instances

    private int x;
    private final String label;

    public Candidate(int x) {
        this.x = x;
        this.label = "candidate#" + x;
        created++;
    }

    public Candidate() {
        this(0);
        reset();
    }

    private Candidate(String label) {  // only reachable from inside the class
        this.x = -1;
        this.label = label;
        created++;
    }

    public int getX() { return x; }

    public void f(int y, double d) {
        if (d < 0) x = y; else x = -y;
    }

    protected void reset() { x = 42; }

    private String secret() { return "the secret of " + label; }

    public static int created() { return created; }

    @Override
    public String toString() {
        return "Candidate{x=" + x + ", label='" + label + "'}";
    }
}
```

stage00-plain/src/main/java/sa4e/reflect/Candidate.java

That is all there is. Three constructors of three visibilities, a private field, a private final field, a static counter, a public method with two parameters, a protected method, a private method, a static method. There is nothing to understand about what `Candidate` *does*, which leaves all your attention for what we do *to* it. Everything else in the book is named after its role in the mechanism: `Inspector`,

`Invoker`, `Validator`, `Interceptor`, `Container`, `Plugin`, `Dependency`. When you later meet Spring's `BeanFactory` or Hibernate's `LazyInitializer`, you will recognise the roles.

`Candidate` changes as little as possible from stage to stage. It gains an annotation here, an interface there, an injected field near the end, and each change is announced. Everything that grows is the framework around it.

How the book and the code fit together

The code lives in a repository with one folder per stage:

```
ReflectAndMore/
  stage00-plain/        stage04-proxies/
  stage01-rtti/         stage05-loading/
  stage02-reflection/   stage06-ioc/
  stage03-annotations/  stage07-codegen/
```

Each folder is a complete, runnable program. All of them use the same package `sa4e.reflect` and the same entry point `sa4e.reflect.Main`, so the difference between two neighbouring folders is exactly one lecture step; `tools/diff-stages.sh 03` shows what stage 03 added to stage 02. Run a stage with nothing but a JDK:

```
./run.sh 03
```

Every program prints Markdown. In the terminal that reads fine; redirected into a file it becomes a document with headings and tables, and each stage folder contains such a file, `output.md`, from the last run. The output excerpts in this book are taken from those files, so what you read here is what the program really printed.

Every chapter follows the same rhythm, because every stage does:

1. **Where we stand.** A concrete limitation of the previous stage. Not a philosophical one; a line of code that cannot be written.
2. **The mechanism.** The JDK API that removes the limitation, in the smallest form that works, walking through the sections of the stage's `Main`.
3. **In the wild.** Which framework does this for a living, and how far our toy is from the real thing.
4. **The bill.** What it costs: performance, security, debuggability, build-time tooling.
5. **Try it yourself.** Exercises. The best way to believe that a container is 150 lines is to add a feature to it.

The map

Stage	Question it answers	Mechanism
00 plain	What does the program look like without any meta-programming?	Nothing yet. Compile-time binding.

01 rtti	What does the JVM know about `Candidate` at run time?	`Class` objects, hierarchy, `Class.forName`, sealed types, records, erasure
02 reflection	Can a program read and operate `Candidate` without knowing it at compile time?	`Inspector`, `Invoker`, private access, the encapsulation wall, `MethodHandle`
03 annotations	How does `Candidate` tell a framework something extra?	`@Comment`, retention, repeatable and inherited annotations, a validator, a test runner
04 proxies	How do we add behaviour around `Candidate` without touching it?	`java.lang.reflect.Proxy`, handler chains, transactions, lazy loading, remote stubs
05 loading	How does code arrive that did not exist at compile time?	Class loaders, configuration, `ServiceLoader`, run-time compilation, class identity
06 ioc	Who creates `Candidate` and fills its dependencies?	A container with scanning, injection, scopes, cycles, an event bus
07 codegen	What if the class itself is written at run time?	ClassFile API, generated subclasses, hidden classes

The stages are cumulative in understanding, not always in code. A helper class travels forward only while a later stage still uses it, so each folder stays focused on its own idea.

A note on tone

We will be unkind to `Candidate`'s privacy. We will read its private fields, call its private methods, change a field it declared `final`, and in the last chapter we will write a subclass of it in raw bytecode without asking. None of this is bad practice in itself; it is *exactly* what Jackson, JUnit, Spring and Hibernate do to your classes every day. The point of doing it by hand once is that you will afterwards know when a framework is doing it to you, and what it means when the JVM says no.

Let us begin where every program begins: with nothing happening at all.

STAGE 00

Plain Java

In which nothing happens, and that is the point.

The program

Stage 00 uses `Candidate` the way every Java program uses every class. It creates two instances, calls a method, prints the results. There is no trick, which makes it the baseline against which every later trick is measured.

```
public final class Main {

    public static void main(String[] args) {
        Report r = Report.stdout();
        r.title("Stage 00: plain Java");

        Candidate a = new Candidate(7);
        Candidate b = new Candidate();
        a.f(3, -1.0);

        r.heading("Using Candidate the ordinary way");
        r.kv("a = new Candidate(7); a.f(3, -1.0)", a);
        r.kv("b = new Candidate()", b);
        r.kv("Candidate.created()", Candidate.created());

        r.heading("What this program cannot do");
        r.item("describe Candidate (its fields, constructors, methods) without
reading its source code");
        r.item("call secret() or the private constructor");
        r.item("create an instance of a class whose name becomes known only while
the program runs");
        r.item("decorate, replace or wire Candidate without editing Main");
        r.flush();
    }
}
```

stage00-plain/src/main/java/sa4e/reflect/Main.java

`Report` is a small helper that prints Markdown: a title, headings, `kv` for an expression and its result, `item` for bullet points. It is identical in every stage and has nothing to do with the material; think of it as `System.out.println` in a nicer suit. The output:

Expression	Result
`a = new Candidate(7); a.f(3, -1.0)`	`Candidate{x=3, label='candidate#7'}`

`b = new Candidate()`	`Candidate{x=42, label='candidate#0'}`
`Candidate.created()`	`2`

Two candidates, one with `x` set by `f` because `d` was negative, one reset to 42 by the no-arg constructor. Nothing you could not have predicted by reading the source. That is the whole point of stage 00: *everything* about this program was decided by reading source.

What the compiler decided

It is worth being precise about what "decided at compile time" means, because the rest of the book is about moving decisions from that moment to a later one.

When `javac` compiles `a.f(3, -1.0)`, it has `Candidate.java` in front of it. It looks up `f`, checks that there is a method of that name taking an `int` and a `double`, checks that `f` is visible from `Main` (it is public), and emits an instruction. Run `javap -c` on the compiled `Main` and you see it:

```
28: aload_2
29: iconst_3
30: ldc2_w        #25     // double -1.0d
33: invokevirtual #27     // Method sa4e/reflect/Candidate.f:(ID)V
```

Three things in these four lines matter for everything that follows.

First, the *check* happened at compile time. If `f` were private, `javac` would have refused, and no class file would exist. Visibility is, first of all, a compiler concept.

Second, the *binding* to the name `f` is still symbolic. The instruction does not say "jump to address 0x4a20"; it says "call the method `f` with descriptor `(ID)V` on class `sa4e/reflect/Candidate`". The class file keeps names. The JVM resolves that name when the instruction first runs, by looking into the loaded `Candidate` class. Keep this in mind: the names survive compilation, which is the reason reflection is possible at all. A C++ compiler would have thrown them away.

Third, the name is a *literal* in `Main`'s constant pool. `Main` can only ever call methods whose names were typed into `Main.java`. It cannot call a method whose name arrives in a configuration file, and it cannot create an instance of a class whose name arrives over the network.

Four things this program cannot do

The second half of `Main` lists them, and each one is a job that some framework does for a living.

Describe Candidate without reading its source. A debugger, a JSON serializer, an object-relational mapper and a test runner all need to know what fields and methods a class has, for a class they have never seen. In stage 00 the only description of `Candidate` is the hand-written `toString()`, which is wrong the moment someone adds a field and forgets to update it. Stages 01 and 02 fix this.

Call `secret()` or the private constructor. The compiler says no, and that is usually right. But a serializer must set private fields when it reads JSON, a test may need to construct an object through a

constructor the author hid, and a dependency-injection container needs to fill a private field without a setter. Stage 02 shows how the door opens, and where the JVM has put a wall.

Create an instance of a class named at run time. `new Candidate(7)` requires the word `Candidate` in the source. A plugin system, a JDBC driver manager, a configuration file saying `implementation=com.acme.FastSorter`, all need to go from a string to an object. Stages 01 and 05 are about that door.

Decorate, replace or wire Candidate without editing Main. To log every call to `f`, Main would have to call a logging wrapper instead. To run `f` inside a transaction, Main would have to begin and commit it. To give `Candidate` a collaborator, Main must create both and hand one to the other. In stage 00 Main does all the wiring by hand, and in a real program Main becomes a thousand lines of `new`. Stages 04 and 06 take that job away from Main.

> **Note**
>
> Notice that none of the four is about `Candidate` being badly designed. `Candidate` is fine. The limitations are limitations of *compile-time binding*: every decision is made by `javac` from source text, and `javac` is long gone when the program runs. Meta-programming is the art of making some of those decisions later, with information that only exists later.

Where the names live

One experiment is worth doing right now, because it foreshadows everything. Compile stage 00 and run the JDK's disassembler on `Candidate` itself:

```
./run.sh 00 > /dev/null          # compiles into stage00-plain/out/classes
javap -p -cp stage00-plain/out/classes sa4e.reflect.Candidate
```

```
public class sa4e.reflect.Candidate {
  private static int created;
  private int x;
  private final java.lang.String label;
  public sa4e.reflect.Candidate(int);
  public sa4e.reflect.Candidate();
  private sa4e.reflect.Candidate(java.lang.String);
  public int getX();
  public void f(int, double);
  protected void reset();
  private java.lang.String secret();
  public static int created();
  public java.lang.String toString();
}
```

Every name, every type, every modifier is in the class file, private ones included. `javap` is not doing anything clever; it reads a well-documented binary format. If a command-line tool can read this, so can a running program, and in the next two stages it will.

> **Takeaways**

- In plain Java, every decision about which class, which method and which field is made by the compiler from source text.
- The compiled class file still contains all names, types and modifiers. Compilation hides nothing; it only fixes the decisions.
- The four things stage 00 cannot do, describing, reaching private members, creating by name, and wiring without editing `Main`, are exactly the jobs of serializers, test runners, plugin systems and containers.

STAGE 01

Run-time type information

In which every object turns out to know exactly what it is, and `Class` turns out to be a thing you can hold in your hand.

Where we stand

Stage 00 could use `Candidate` but could not say anything *about* it. Given a variable of type `Candidate`, Main could not find out whether the object was really a `Candidate` or a subclass; it could not compare two types; it could not take the string `"sa4e.reflect.Candidate"` and turn it into an object. All of that is compile-time knowledge, and at run time it was gone.

Except that it is not gone. This chapter is about the fact that the JVM keeps a complete description of every type, that this description is itself an object, and that you can ask it questions. Nothing is read or changed yet; that is stage 02. Stage 01 only looks.

Every object knows its class

Here is the fact that makes everything else possible. Every object on the JVM heap starts with a header, and the header contains a pointer to the object's class. Not a type tag, not an integer, a pointer to a full description: name, superclass, interfaces, fields, methods, constant pool, and a reference to a `java.lang.Class` object that represents the class at the Java level. `getClass()` follows that pointer. It is one of the cheapest calls in the language, and it never lies.

Stage 01 adds a subclass so that there is something to lie about:

```
public class Special extends Candidate implements Comparable<Special> {
    public Special(int x) { super(x); }

    @Override
    public int compareTo(Special other) {
        return Integer.compare(getX(), other.getX());
    }
}
```

and asks the obvious question:

```
Candidate c = new Special(7);        // static type Candidate, dynamic type Special
r.kv("c.getClass()", c.getClass());
r.kv("c.getClass().getName()", c.getClass().getName());
r.kv("c instanceof Special", c instanceof Special);
```

Expression	Result
`Candidate c = new Special(7)`	`declared type Candidate`
`c.getClass()`	`class sa4e.reflect.Special`
`c.getClass().getName()`	`sa4e.reflect.Special`
`c instanceof Special`	`true`

The variable `c` has the *static* type `Candidate`: that is what the compiler knows and what it will let you call. The object has the *dynamic* type `Special`: that is what the JVM knows. `instanceof` and virtual method dispatch have always used the dynamic type; `getClass()` simply hands it to you.

Class objects are ordinary values

`c.getClass()` returns a `Class<?>`, and `Class` is a perfectly ordinary Java class. Its instances can be stored in variables, put in maps, passed to methods, compared with `==`. There is a literal syntax for them, `Candidate.class`, and there is one for every type in the language, including the ones that are not classes at all:

```
Class<?>[] samples = {int.class, Integer.class, int[].class, String[][].class,
        Candidate.class, Special.class, Variant.class, Variant.A.class,
        Variant.Mode.class, Runnable.class, void.class};
```

getName()	getSimpleName()	getCanonicalName()	kind
`int`	int	int	primitive
`java.lang.Integer`	Integer	java.lang.Integer	class
`[I`	int[]	int[]	array of int
`[[Ljava.lang.String;`	String[][]	java.lang.String[][]	array of String[]
`sa4e.reflect.Candidate`	Candidate	sa4e.reflect.Candidate	class
`sa4e.reflect.Special`	Special	sa4e.reflect.Special	class
`sa4e.reflect.Variant`	Variant	sa4e.reflect.Variant	sealed interface
`sa4e.reflect.Variant$A`	A	sa4e.reflect.Variant.A	record
`sa4e.reflect.Variant$Mode`	Mode	sa4e.reflect.Variant.Mode	enum
`java.lang.Runnable`	Runnable	java.lang.Runnable	interface
`void`	void	void	primitive

A few things in this table surprise people the first time.

`int.class` exists and is not `Integer.class`. Primitives have Class objects so that reflection can describe a method like `f(int, double)` precisely; a parameter of type `int` and a parameter of type `Integer` are different things, and the JVM must know which one to box.

Arrays are classes, with names that look like line noise. `[I` is "array of int", `[[Ljava.lang.String;` is "array of array of String". These are the JVM's internal descriptors leaking through. `getSimpleName()` gives the human form, `getName()` gives the form that `Class.forName()` understands.

Nested classes use `$` in `getName()` and `.` in `getCanonicalName()`. `Variant$A` is the real name as far as the JVM and the class file are concerned; `Variant.A` is the Java-language spelling. Frameworks that take class names from configuration files must accept the `$` form, and the error messages of frameworks that do not are a small genre of their own.

Even `void` has a Class object, so that `getReturnType()` can return *something* for a `void` method.

There is exactly one `Class` object per loaded class, which is why `==` works:

Expression	Result
`Candidate.class == c.getClass()`	`false`
`Candidate.class == new Candidate().getClass()`	`true`

The first line is `false` because `c` is a `Special`. A `Class` object identifies a type exactly; it does not know about "is a kind of". For that there is `isAssignableFrom`, further down.

From a string to a type

Now the door that everything in the second half of the book walks through:

```
Class<?> byName = Class.forName("sa4e.reflect.Candidate");
Object instance = byName.getDeclaredConstructor().newInstance();
```

Expression	Result
`Class.forName("sa4e.reflect.Candidate")`	`class sa4e.reflect.Candidate`
`byName.getDeclaredConstructor().newInstance()`	`Candidate{x=42, label='candidate#0'}`
`Class.forName("Candidate")`	`ClassNotFoundException (the package is part of the name)`

Read the first two lines slowly. The word `Candidate` does not appear as a type anywhere in them. A *string* went in, an *object* came out, and the string could have come from a properties file, a command-line argument, a database row or a network packet. This is how a JDBC driver manager finds `org.postgresql.Driver`, how a logging facade finds its back-end, how a plugin system finds plugins, and how a dependency-injection container finds everything. We are getting ahead of ourselves, and `getDeclaredConstructor().newInstance()` is really stage 02 material, but it was impossible to resist showing the door open.

The third line is the first error of the book and a common one: `Class.forName` wants the fully qualified name. There is no "current package" at run time; packages are part of the name, full stop.

> **Note**
>
> `Class.forName(name)` does three things: it *loads* the class if it is not loaded yet, it *links* it, and it *initializes* it, which means running its static initializers. The third step is a side effect,

> and it is deliberate: for many years JDBC drivers registered themselves in a static initializer, and `Class.forName("org.postgresql.Driver")` was the idiom to trigger it. Stage 05 takes this apart.

Walking the hierarchy

A `Class` object knows where it sits:

Expression	Result
`superclass chain of Special`	`Special -> Candidate -> Object`
`Special.class.getInterfaces()`	`[interface java.lang.Comparable]`
`all interfaces of ArrayList (transitively)`	`[List, SequencedCollection, Collection, Iterable, RandomAccess, Cloneable, Serializable]`
`Object.class.getSuperclass()`	`null`

`getSuperclass()` returns one class or `null` at the top; `getInterfaces()` returns only the interfaces this class declares directly, so "all interfaces of ArrayList" needs a little recursion over superclasses and super-interfaces. The loop that produced the first line is three lines of code:

```
for (Class<?> k = Special.class; k != null; k = k.getSuperclass()) {
    chain.append(k.getSimpleName()).append(k.getSuperclass() != null ? " -> " : "");
}
```

Every serializer walks exactly this loop to find the fields of superclasses, and every container walks it to find annotations on them.

Type tests and casts, without the syntax

Everything `instanceof` and the cast operator do is also available as a method call on a `Class` object, which matters when the class is a variable rather than a name in the source:

Expression	Result
`Candidate.class.isAssignableFrom(Special.class)`	`true`
`Special.class.isAssignableFrom(Candidate.class)`	`false`
`Special.class.isInstance(c)`	`true`
`Special.class.cast(c).getX()`	`7`
`Variant.A.class.cast(c)`	`ClassCastException`

`a.isAssignableFrom(b)` reads as "a variable of type `a` can hold a `b`". The direction trips everyone up at least once; the mnemonic is that it mirrors an assignment `a = b`. `isInstance` is `instanceof` with the type on the left, and `cast` is the cast operator, including the `ClassCastException` when it fails. The one new thing `cast` gives you is a *typed* return value: `Special.class.cast(c)` is a `Special` as far as the compiler is concerned, so you can call `getX()` on it directly.

When stage 06 asks "which registered classes implement `Capability`?", the answer is one `isAssignableFrom` per registered class. That is the whole algorithm.

The compiler uses RTTI too

Modern Java has pattern matching, and it is worth pausing to notice that it is built on the same foundation:

```
public sealed interface Variant permits Variant.A, Variant.B {
    record A(int value, String tag) implements Variant {}
    final class B implements Variant { /* ... */ }
    enum Mode { ON, OFF, UNKNOWN }
}
```

```
String text = switch (v) {                  // no default branch needed
    case Variant.A a -> "A with value " + a.value() + " and tag " + a.tag();
    case Variant.B b -> "B with weight " + b.weight();
};
```

Expression	Result
`switch (v) for a A`	`A with value 1 and tag one`
`switch (v) for a B`	`B with weight 2.5`

The switch compiles to type tests against the object's class, the same test `isInstance` performs. And because `Variant` is *sealed*, the compiler knows that `A` and `B` are the only possibilities and does not demand a `default`. Sealed types are RTTI that the compiler can reason about; the next section shows that the run time can see it as well.

Types that describe themselves

Sealed interfaces, records and enums all carry extra structure, and all of it is visible at run time:

Expression	Result
`Variant.class.isSealed()`	`true`
`Variant.class.getPermittedSubclasses()`	`[A, B]`
`Variant.A.class.isRecord()`	`true`
`Variant.A.class.getRecordComponents()`	`value: int, tag: String`
`Variant.B.class.isRecord()`	`false`
`Variant.Mode.class.isEnum()`	`true`
`Variant.Mode.class.getEnumConstants()`	`[ON, OFF, UNKNOWN]`

`getRecordComponents()` deserves a highlight. A record declares its components once, in the header, and the JVM remembers them in order, with names and types. A serializer that meets a record does not have to guess which fields matter or which constructor to call: the components are the fields, the canonical constructor takes them in exactly this order, and the accessors have exactly these names. Records were designed, among other things, to make reflection boring, and stage 02 will make use of that.

Generics: erased from objects, kept in declarations

Here is the one place where the "every object knows its class" story has a hole:

Expression	Result
`new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()`	`true`

The two lists have the same class. Type arguments are erased when the compiler is done with them; at run time there is only `ArrayList`. This is the famous *type erasure*, and it is the reason you cannot write `new T()` or `list instanceof List<String>`.

But erasure applies to *objects*, not to *declarations*. A field declared as `List<String>`, a method returning `Map<String, List<Integer>>`, a class extending `TypeRef<List<String>>`: these declarations keep their full generic signature in the class file, and reflection can read it back:

```
static class Holder {
    List<String> items = new ArrayList<>();
    Map<String, List<Integer>> index = Map.of();
}
```

Expression	Result
`field items: getType()`	`List`
`field items: getGenericType()`	`java.util.List<java.lang.String>`
`field index: getType()`	`Map`
`field index: getGenericType()`	`java.util.Map<java.lang.String, java.util.List<java.lang.Integer>>`

`getType()` is the erased class; `getGenericType()` is the declaration, as a `java.lang.reflect.Type` that can be a `ParameterizedType` with actual type arguments. This is how Jackson knows that the field `items` should be filled with strings, not with `Object`s, when it reads JSON.

And then there is the trick that every Java developer should see once, because it looks like black magic and is not:

```
public abstract class TypeRef<T> {
    private final Type type;

    protected TypeRef() {
        Type superclass = getClass().getGenericSuperclass();
        if (!(superclass instanceof ParameterizedType parameterized)) {
            throw new IllegalStateException("use: new TypeRef<SomeType>() {}");
        }
        this.type = parameterized.getActualTypeArguments()[0];
    }

    public Type type() { return type; }
}
```

```
TypeRef<List<String>> ref = new TypeRef<>() {};
r.kv("new TypeRef<List<String>>() {}.type()", ref.type());
```

Expression	Result
`new TypeRef<List<String>>() {}.type()`	`java.util.List<java.lang.String>`

The `{}` at the end is the whole trick. It creates an *anonymous subclass* of `TypeRef<List<String>>`, and that subclass is a declaration, so its generic superclass survives erasure. The constructor reads it

back. Jackson's `TypeReference`, Gson's `TypeToken` and Guice's `TypeLiteral` are this class, and now you know why you have to write those curly braces.

Lambdas and anonymous classes have classes too

The last section is a teaser for stage 07:

Expression	Result
`lambda.getClass().getName()`	`sa4e.reflect.Main$$Lambda/0x0000018001046660`
`lambda.getClass().isHidden()`	`true`
`anonymous.getClass().getName()`	`sa4e.reflect.Main$2`
`anonymous.getClass().isAnonymousClass()`	`true`

An anonymous class is compiled by `javac` into an ordinary class file called `Main$2.class`. A lambda is not: there is no class file for it anywhere. The class `Main$$Lambda/0x...` is generated by the JVM the first time the lambda expression runs, and it is *hidden*: it has no findable name, `Class.forName` cannot see it, and it disappears when it is no longer referenced. Stage 07 generates classes exactly like this, so remember the word "hidden".

In the wild

- Every DI container starts with `isAssignableFrom` to find candidates for an interface, and with `Class.forName` to turn configuration into classes.
- JDBC (`Class.forName("org.postgresql.Driver")`), JNDI, logging back-ends, JAX-RS runtimes and plugin systems all begin with a class name in a string.
- Jackson's `TypeReference<List<Customer>>`, Gson's `TypeToken` and Guice's `TypeLiteral` are `TypeRef`.
- Hibernate reads `getGenericType()` of every collection-typed field to know what an `@OneToMany` contains.

The bill

Almost nothing. `getClass()` is a pointer dereference, `isInstance` is the same check as `instanceof`, and the `Class` objects already exist because the JVM needs them anyway. The one expensive and side-effect-laden call is `Class.forName`, which may load and initialize a class; stage 05 is about exactly that.

Try it yourself

1. Print the full inheritance chain of `java.util.LinkedHashMap` including all interfaces, transitively, without duplicates.
2. Write `boolean sameHierarchy(Object a, Object b)` that returns true when one object's class is a subclass of the other's, without using `instanceof`.
3. Make `TypeRef` work for nested arguments such as `Map<String, List<Integer>>` and print the inner types recursively. (Hint: `ParameterizedType` is itself a `Type`.)

Takeaways

- Every object knows its class; `getClass()` returns the dynamic type, the one the JVM uses for dispatch.
- `Class` objects are values: one per loaded type, comparable with `==`, obtainable from a literal, from an object or from a string.
- `isAssignableFrom`, `isInstance` and `cast` are `instanceof` and the cast operator for cases where the type is a variable.
- Sealed types, records and enums describe their own structure, which makes them easy for frameworks.
- Generics are erased from objects but kept in declarations; the super type token exploits exactly that.

STAGE 02

Reflection

In which we read `Candidate` like a book, call what we are not supposed to call, and discover that the JDK has built itself a wall.

Where we stand

Stage 01 could ask a class *what* it is. It could not ask what the class *contains*, and it could not do anything with the answer: no object creation with arguments, no method call by name, no access to a field unless the field's name was typed into the source. The `Inspector` that this chapter builds answers the first question for any class in the world, and the second half of the chapter does the rest: create, call, read, write, by name, at run time.

The package is `java.lang.reflect`, and its central idea is that every *member* of a class is itself an object. A `Constructor`, a `Field`, a `Method`, a `Parameter`: each can be listed, inspected and used. The `Class` object from stage 01 is the index; these are the entries.

Inspector: describing a class you have never seen

The lecture slides call this example "TalkAbout". It takes a `Class<?>` and produces an XML description of it, and it does so without instantiating the class, without importing it, and without knowing it at compile time:

```
public final class Inspector {
    private final Class<?> target;

    public Inspector(Class<?> target) { this.target = target; }

    public String describe() {
        StringBuilder xml = new StringBuilder();
        xml.append("<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n");
        xml.append("<class name=\"").append(target.getName())
           .append("\"
modifiers=\"").append(Modifier.toString(target.getModifiers()))
           .append("\" superclass=\"").append(target.getSuperclass().getName())
           .append("\">\n");

        // The reflection API makes no promise about the order of members, so sort.
        Constructor<?>[] constructors = target.getDeclaredConstructors();
```

```
        Arrays.sort(constructors,
Comparator.comparing(Constructor::toGenericString));
        for (Constructor<?> ctor : constructors) {
            xml.append("  <constructor modifiers=\"")
               .append(Modifier.toString(ctor.getModifiers())).append("\">\n");
            appendParameters(xml, ctor);
            xml.append("  </constructor>\n");
        }

        Field[] fields = target.getDeclaredFields();
        Arrays.sort(fields, Comparator.comparing(Field::getName));
        for (Field f : fields) {
            xml.append("  <field name=\"").append(f.getName())
               .append("\" type=\"").append(f.getType().getName())
               .append("\"
modifiers=\"").append(Modifier.toString(f.getModifiers()))
               .append("\"/>\n");
        }

        Method[] methods = target.getDeclaredMethods();
        Arrays.sort(methods, Comparator.comparing(Method::getName));
        for (Method m : methods) {
            if (m.isSynthetic()) continue;        // compiler-generated helpers
            xml.append("  <method name=\"").append(m.getName())
               .append("\"
modifiers=\"").append(Modifier.toString(m.getModifiers()))
               .append("\"
returns=\"").append(m.getReturnType().getName()).append("\">\n");
            appendParameters(xml, m);
            xml.append("  </method>\n");
        }
        xml.append("</class>\n");
        return xml.toString();
    }

    /** Constructors and methods share the type Executable, so parameters are
handled once. */
    private static void appendParameters(StringBuilder xml, Executable executable) {
        for (Parameter p : executable.getParameters()) {
            xml.append("    <parameter name=\"").append(p.getName())
               .append("\" type=\"").append(p.getType().getName()).append("\"/>\n");
        }
    }
}
```

stage02-reflection/src/main/java/sa4e/reflect/Inspector.java (abridged)

Three families of calls do all the work: `getDeclaredConstructors()`, `getDeclaredFields()` and `getDeclaredMethods()`. Each returns an array of member objects, and each member object can tell its name, its modifiers as a bit mask that `Modifier.toString` turns into `private static final`, and its types. Constructors and methods share a superclass, `Executable`, so parameters are handled once for both. Applied to `Candidate`, the result is:

```
<?xml version="1.0" encoding="UTF-8"?>
<class name="sa4e.reflect.Candidate" modifiers="public"
superclass="java.lang.Object">
  <constructor modifiers="private">
    <parameter name="label" type="java.lang.String"/>
```

```
  </constructor>
  <constructor modifiers="public">
  </constructor>
  <constructor modifiers="public">
    <parameter name="x" type="int"/>
  </constructor>
  <field name="created" type="int" modifiers="private static"/>
  <field name="label" type="java.lang.String" modifiers="private final"/>
  <field name="x" type="int" modifiers="private"/>
  <method name="created" modifiers="public static" returns="int">
  </method>
  <method name="f" modifiers="public" returns="void">
    <parameter name="y" type="int"/>
    <parameter name="d" type="double"/>
  </method>
  <method name="getX" modifiers="public" returns="int">
  </method>
  <method name="reset" modifiers="protected" returns="void">
  </method>
  <method name="secret" modifiers="private" returns="java.lang.String">
  </method>
  <method name="toString" modifiers="public" returns="java.lang.String">
  </method>
</class>
```

Compare this with the `javap` listing at the end of stage 00. It is the same information, produced by a running program instead of a command-line tool. The private constructor, the private field, the private method: all there. Reflection does not respect visibility when *listing*; visibility only matters when you try to *use* a member, and even then, as we will see, only up to a point.

Declared versus inherited

The word "Declared" in those method names is not decoration. `getDeclaredMethods()` returns every method *declared in this class*, of any visibility, and nothing inherited. `getMethods()` returns every *public* method, including inherited ones, all the way up to `Object`:

Expression	Result
`Candidate.class.getDeclaredMethods().length`	`6 (all methods declared in Candidate, any visibility)`
`Candidate.class.getMethods().length`	`12 (public methods only, but including those inherited from Object)`
`Special.class.getDeclaredMethods().length`	`2 (compareTo plus a synthetic bridge method generated for Comparable<Special>)`

Six declared methods in `Candidate`: `created`, `f`, `getX`, `reset`, `secret`, `toString`. Twelve public methods: `created`, `f`, `getX`, `toString`, plus `equals`, `hashCode`, `getClass`, `notify`, `notifyAll` and three overloads of `wait` from `Object`. Neither view is "the" list of methods; which one you want depends on what you are doing. A serializer wants declared fields of the class *and* of its superclasses, so it walks the superclass chain from stage 01 and calls `getDeclaredFields()` on each.

The third line hides a small lesson. `Special` declares one method, `compareTo(Special)`, but the JVM sees two. Because `Comparable<T>` erases to `compareTo(Object)`, the compiler generates a *bridge*

method with that signature that casts and delegates. It is marked `synthetic`, and `Inspector` skips synthetic members for that reason. Expect them; every generic class has some.

Parameter names, if you ask nicely

The XML above shows `<parameter name="y" type="int"/>`. That is less obvious than it looks:

Expression	Result
`parameter of f: y : int`	`isNamePresent=true`
`parameter of f: d : double`	`isNamePresent=true`

Parameter names are not in the class file by default. The JVM does not need them, so `javac` throws them away unless it is told to keep them with the flag `-parameters`. Without that flag, `Parameter.getName()` returns `arg0`, `arg1`, and `isNamePresent()` returns `false`. This project compiles with the flag because frameworks depend on it: Spring MVC maps a request parameter `?x=5` to a method parameter called `x` by name, and Jakarta REST and most JSON binders for constructors do the same. If you ever see `arg0` in an error message from such a framework, you now know which flag is missing.

Creating objects by name

The second half of the chapter stops looking and starts touching. First, construction:

```
Class<?> type = Class.forName("sa4e.reflect.Candidate");
Object viaPublic = type.getConstructor(int.class).newInstance(7);

Constructor<?> hidden = type.getDeclaredConstructor(String.class);
hidden.setAccessible(true);                      // private: this is the key
Object viaPrivate = hidden.newInstance("made by reflection");
```

Expression	Result
`type.getConstructor(int.class).newInstance(7)`	`Candidate{x=7, label='candidate#7'}`
`private constructor, after setAccessible(true)`	`Candidate{x=-1, label='made by reflection'}`

`getConstructor(int.class)` selects a constructor by its parameter types, exactly as the compiler would, and `newInstance(7)` calls it with boxed arguments. `getConstructor` sees only public constructors; `getDeclaredConstructor` sees all of them. And then the line that the whole chapter has been building up to: `setAccessible(true)`.

`setAccessible(true)` asks the reflection API to skip the language-level access check for this one member. After it, a private constructor can be called, a private method invoked, a private field read and written. It is the single most consequential method in `java.lang.reflect`, and every framework in the book's "In the wild" boxes calls it. It is also the method that Java 16 decided to say no to, in some circumstances, and we will get to that.

Calling methods by name

```
Candidate c = new Candidate(1);
Candidate.class.getMethod("f", int.class, double.class).invoke(c, 3, -1.0);

Method secret = Candidate.class.getDeclaredMethod("secret");
secret.setAccessible(true);
Object s = secret.invoke(c);

Method created = Candidate.class.getMethod("created");
Object n = created.invoke(null);                  // static: no target object
```

Expression	Result
`getMethod("f", int.class, double.class).invoke(c, 3, -1.0)`	`Candidate{x=3, label='candidate#1'}`
`private secret() after setAccessible(true)`	`the secret of candidate#1`
`static created(), invoked with target null`	`3`
`Integer.parseInt("forty-two") via reflection`	`InvocationTargetException wrapping java.lang.NumberFormatException: For input string: "forty-two"`
`secret.invoke("not a Candidate")`	`IllegalArgumentException: object of type java.lang.String is not an instance of sa4e.reflect.Candidate`

`Method.invoke(target, args...)` is the universal call. The target is the object to call the method on, or `null` for a static method. Arguments travel as `Object`, so `3` is boxed to an `Integer` on the way in and unboxed to an `int` by the JVM on the way out; the return value comes back boxed as well. Everything the compiler would have checked at compile time is now checked at run time, and the last two lines show what that looks like.

The fourth line is the one that bites everyone once. When the *called method* throws, `invoke` does not rethrow that exception. It wraps it in an `InvocationTargetException` whose `getCause()` is the real one. The reason is sound: `invoke` must distinguish "I could not perform the call" (wrong argument count, no access) from "the call ran and threw", and Java's checked-exception rules leave no other way. The consequence is that every piece of reflective code contains a `catch (InvocationTargetException e) { throw e.getCause(); }` somewhere, and so will ours in stage 04.

Reading and writing fields, even final ones

```
Field x = Candidate.class.getDeclaredField("x");
Field label = Candidate.class.getDeclaredField("label");
x.setAccessible(true);
label.setAccessible(true);

label.set(c, "changed although final");     // allowed on ordinary instance fields
```

```
Field counter = Candidate.class.getDeclaredField("created");
counter.setAccessible(true);
counter.setInt(null, 1000);                  // static field: target is null
```

Expression	Result
`x.getInt(c)`	`3`
`label.get(c)`	`candidate#1`
`label.set(c, "changed although final")`	`Candidate{x=3, label='changed although final'}`
`counter.setInt(null, 1000); Candidate.created()`	`1000`
`value.setInt(record, 2) on a record`	`IllegalAccessException: Can not set final int field sa4e.reflect.Variant$A.value to (int)2`

The third line should make you slightly uncomfortable. `label` is declared `private final`, and we changed it. For instance fields of ordinary classes, `setAccessible(true)` lifts the `final` as well as the `private`. This is not an oversight; serialization frameworks need it. When Jackson deserializes JSON into an object with final fields and no suitable constructor, this is how it does it, and `java.io.ObjectInputStream` has done the same since 1997.

The fifth line is where the JVM draws a line of its own. Fields of a *record* cannot be set this way, not even with `setAccessible(true)`; neither can `static final` fields or fields of hidden classes. Records promised immutability as part of their contract, and the JVM keeps that promise against reflection. A serializer that meets a record must go through the canonical constructor, which is exactly what `getRecordComponents()` from stage 01 makes easy.

The wall

Everything so far worked on our own classes. Now try it on the JDK:

```
Field stringValue = String.class.getDeclaredField("value");
stringValue.setAccessible(true);             // throws
```

Expression	Result
`String.class.getDeclaredField("value")`	`private final byte[] java.lang.String.value (reading metadata always works)`
`stringValue.setAccessible(true)`	`InaccessibleObjectException: Unable to make field private final byte[] java.lang.String.value accessible: module java.base does not "opens java.lang" to unnamed module @5b6f7412`

Listing the field works, as it always does; metadata is never hidden. But `setAccessible(true)` throws an `InaccessibleObjectException`, and the message tells the whole story: the module `java.base` does not *open* the package `java.lang` to our code.

This is *strong encapsulation*, introduced with the module system in Java 9 and enforced by default since Java 16. The JDK's own packages are closed to deep reflection unless a module explicitly opens

them, and `java.base` does not. Before Java 16, the line above would have worked, and a dozen popular libraries used it to mutate strings, poke at `Unsafe` or rewrite class loaders. When Java 16 closed the door, those libraries broke, their authors complained, and the JDK team held firm, with one concession: the JVM flag

```
java --add-opens java.base/java.lang=ALL-UNNAMED ...
```

opens a package at launch time for code that genuinely needs it. If you have ever wondered why a Gradle build or an application server starts with a dozen `--add-opens` flags, this is why.

> **Note**
>
> Your own code can build the same wall. A `module-info.java` that does not `opens` a package makes that package's private members inaccessible to reflection from outside, even with `setAccessible(true)`. Frameworks that need deep reflection ask you to `opens your.package to spring.core;`. Without a module descriptor, as in this project, your classes live in the "unnamed module", which is open to everyone, which is why every example in this book just works.

Arrays

Arrays are classes, so of course there is reflection for them:

```
Object array = Array.newInstance(Candidate.class, 2);   // a Candidate[2]
Array.set(array, 0, new Candidate(5));
Array.set(array, 1, viaPrivate);
```

Expression	Result
`Array.newInstance(Candidate.class, 2)`	`Candidate[] of length 2`
`Array.get(array, 1)`	`Candidate{x=-1, label='made by reflection'}`

`java.lang.reflect.Array` creates and accesses arrays whose component type is a variable. It exists because `new T[n]` is impossible with erased generics, and because a deserializer that reads a JSON array has only a `Class<?>` to say what kind of array to build.

Invoker: a convenience layer

The raw API asks for exact parameter types: `getMethod("f", int.class, double.class)`. That is precise but tedious, and it is not what a scripting engine or a test framework wants. They have a name and some argument values, and they want the member that fits. `Invoker` is forty lines that do that:

```
public static Object call(Object target, String name, Object... args)
        throws ReflectiveOperationException {
    boolean isStatic = target instanceof Class<?>;
    Class<?> type = isStatic ? (Class<?>) target : target.getClass();
    for (Class<?> k = type; k != null; k = k.getSuperclass()) {
```

```
        for (Method m : k.getDeclaredMethods()) {
            if (m.getName().equals(name) && matches(m.getParameterTypes(), args)) {
                m.setAccessible(true);
                return m.invoke(isStatic ? null : target, args);
            }
        }
    }
    throw new NoSuchMethodException(type.getName() + "." + name);
}

/** True if every argument can be passed to the corresponding parameter (boxing
included). */
private static boolean matches(Class<?>[] parameterTypes, Object[] args) {
    if (parameterTypes.length != args.length) return false;
    for (int i = 0; i < args.length; i++) {
        if (args[i] == null) {
            if (parameterTypes[i].isPrimitive()) return false;   // null cannot
become an int
            continue;
        }
        if (!wrap(parameterTypes[i]).isInstance(args[i])) return false;
    }
    return true;
}

/** int.class -> Integer.class etc.; non-primitive types are returned unchanged. */
private static Class<?> wrap(Class<?> type) {
    return type.isPrimitive() ? MethodType.methodType(type).wrap().returnType() :
type;
}
```

stage02-reflection/src/main/java/sa4e/reflect/Invoker.java (abridged)

Every piece of stage 01 is in there: the superclass walk, `isInstance`, `isPrimitive`. The `wrap` helper is a small trick worth stealing: `MethodType` from `java.lang.invoke` knows how to map `int.class` to `Integer.class`, so nobody has to write the eight-entry table by hand.

Expression	Result
`Invoker.newInstance("sa4e.reflect.Candidate", 9)`	`Candidate{x=9, label='candidate#9'}`
`Invoker.call(made, "secret")`	`the secret of candidate#9`
`Invoker.call(made, "f", 4, 1.5)`	`Candidate{x=-4, label='candidate#9'}`
`Invoker.call(Candidate.class, "created")`	`1002`
`Invoker.set(made, "x", 77); Invoker.get(made, "x")`	`77`
`Invoker.call(new Special(3), "secret")`	`the secret of candidate#3 (inherited private method)`

Notice the last line: `secret()` is private to `Candidate`, and `Invoker` found and called it on a `Special`. The compiler would never have allowed that. Notice also what we paid for the convenience: method names are now strings. The compiler cannot check them, and when someone renames `secret` to `hint`, the IDE's refactoring will not follow the string. Every framework that identifies members by name lives with this, which is one reason the next stage introduces annotations.

What does it cost?

The last section of stage 02 measures. Twenty million calls of `getX()` on four different objects, after three warm-up rounds so that the JIT compiler has done its work:

```
Method m = Candidate.class.getMethod("getX");
MethodHandle mh = MethodHandles.lookup()
        .findVirtual(Candidate.class, "getX", MethodType.methodType(int.class));

for (int i = 0; i < n; i++) sum += targets[i & 3].getX();              // direct
for (int i = 0; i < n; i++) sum += (int) m.invoke(targets[i & 3]);     // reflection
for (int i = 0; i < n; i++) sum += (int) mh.invokeExact(targets[i & 3]); // method
handle
```

Call style	ns per call
direct call	0.35
`Method.invoke`	6.27
`MethodHandle.invokeExact`	2.37

These numbers are from one laptop and one run, with no benchmark harness, so treat them as a shape, not a measurement. The shape is reliable: a reflective call costs an order of magnitude more than a direct call, because arguments are boxed into an `Object[]`, access is checked, and the JIT cannot inline through `invoke`. The third row introduces `java.lang.invoke.MethodHandle`, a lower-level mechanism from Java 7 that is typed, does no boxing with `invokeExact`, and sits much closer to a direct call. Since Java 18, `Method.invoke` is itself implemented on top of method handles (JEP 416), which is why the gap is smaller than it used to be.

The practical rule is simple. Frameworks do reflection *once*, at startup, to find out what to do, and then do it millions of times through something cheaper: a cached `Method`, a `MethodHandle`, a generated class (stage 07). Reflection in a hot loop is a bug.

In the wild

- Jackson and Gson read and write private fields exactly as in the fields section, final ones included.
- JUnit finds and invokes test methods as in the methods section; stage 03 adds the annotations that tell it which ones.
- Spring and Jakarta CDI call constructors by reflection as in the construction section, with arguments they resolved themselves (stage 06).
- Mockito, Hibernate and most serialization frameworks needed `--add-opens` flags or rewrites when Java 16 enforced the wall.

The bill

- Member names are strings: no compile-time check, no refactoring support, errors at run time.
- A reflective call is several times slower than a direct one; cache members, use method handles, never reflect in a loop.

- Reflection defeats encapsulation. The JDK protects itself since Java 16; your own modules can do the same.
- GraalVM native image compiles with a closed world and cannot see the strings; everything accessed reflectively must be listed in `reachability-metadata.json`, or it is simply not there at run time.

Try it yourself

1. Let `Inspector` emit JSON instead of XML, and add the inherited public methods (`getMethods()`) as a separate section.
2. `Invoker.call(made, "f", 4, 1)` fails, because `1` is an `Integer` and the parameter is `double`. Add widening conversions.
3. Run stage 02 with `./run.sh 02 --add-opens java.base/java.lang=ALL-UNNAMED` and change the content of a `String` in place. Then explain why string literals elsewhere in the program may change too. (Hint: the string pool.)
4. Replace `Method.invoke` in the benchmark by a lambda created with `LambdaMetafactory` and compare again.

Takeaways

- Every member of a class is an object: `Constructor`, `Field`, `Method`, `Parameter`. Listing them ignores visibility; using them respects it, until `setAccessible(true)`.
- `invoke` wraps exceptions from the called method in `InvocationTargetException`; always unwrap.
- Final instance fields of ordinary classes can be changed; record fields, static finals and hidden classes cannot.
- The JDK is closed to deep reflection since Java 16; `--add-opens` is the escape hatch.
- Reflect once, cache, and call through something cheaper.

STAGE 03

Annotations

In which `Candidate` *finally gets to talk back, and we write two frameworks before lunch.*

Where we stand

Reflection, as of stage 02, is a one-way conversation. A framework can read everything about a class, but the class cannot say anything to the framework. Which field must never be null? Which method is a test? Which parameter has a valid range? Which constructor should a container use? In stage 02 such knowledge would have to live in the framework's code, in naming conventions (`testSomething`, `setSomething`) or in XML files kept somewhere else, going stale somewhere else.

Annotations are the answer: structured metadata, written *next to* the thing it describes, checked by the compiler, and, with the right retention, readable through reflection. The first half of this chapter is about the mechanism. The second half builds two small frameworks on it, a validator and a test runner, and in doing so introduces the idea that will carry the rest of the book: inversion of control.

Defining an annotation and reading it back

An annotation type is declared with `@interface`, and it is decorated with two meta-annotations that answer two questions: *where may it be written* and *how long does it live*:

```
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.CONSTRUCTOR, ElementType.FIELD,
         ElementType.METHOD, ElementType.PARAMETER})
public @interface Comment {
    String value() default "";
}
```

stage03-annotations/src/main/java/sa4e/reflect/Comment.java

`@Target` lists the kinds of elements that may carry the annotation; the compiler rejects `@Comment` on a local variable because `LOCAL_VARIABLE` is not in the list. `@Retention(RUNTIME)` says the annotation survives into the class file *and* is visible to reflection, which is what we want here. The body declares *members* as parameterless methods; `value()` is special in that `@Comment("text")` is shorthand for `@Comment(value = "text")`.

`Candidate` now wears a few of these:

```
@Comment("the class under investigation")
@Marker
public class Candidate {

    @Range(min = -100, max = 100)
    private int x;

    @NotNull
    private final String label;

    @Comment("reads x")
    public int getX() { return x; }

    public void f(@Range(min = -10, max = 10) int y, double d) { /* unchanged */ }
    // ...
}
```

Nothing about `Candidate`'s behaviour changed. Annotations are inert: they are claims, and nothing happens unless someone reads them. Reading is one method, `getAnnotation`, available on everything that can be annotated:

Expression	Result
`Candidate.class.getAnnotation(Comment.class).value()`	`the class under investigation`
`getX.getAnnotation(Comment.class).value()`	`reads x`
`Candidate.class.getMethod("f", ...).isAnnotationPresent(Comment.class)`	`false`
`Candidate.class.getMethod("reset")`	`NoSuchMethodException (getMethod sees public methods only; use getDeclaredMethod)`
`parameter y of f: getAnnotation(Range.class)`	`min=-10, max=10`

`Class`, `Constructor`, `Field`, `Method` and `Parameter` all implement `AnnotatedElement`, so the same `getAnnotation(Type.class)` works everywhere and returns `null` when the annotation is absent. The return type is the annotation type itself, so `.value()` is a normal, type-checked method call. Compare that with the stringly-typed `Invoker` of stage 02: annotations give the framework a vocabulary the compiler understands.

Meta-annotations: annotations all the way down

`@Retention` and `@Target` are themselves annotations, on the annotation type, and they can be read like any other:

Expression	Result
`Comment.class.isAnnotation()`	`true`
`Comment.class.getAnnotation(Retention.class).value()`	`RUNTIME`

`Comment.class.getAnnotation(Target.class).value()`	`[TYPE, CONSTRUCTOR, FIELD, METHOD, PARAMETER]`
`Comment.class.getDeclaredMethods()`	`[public abstract java.lang.String sa4e.reflect.Comment.value()]`

The last line is the one to remember: an annotation type is an *interface*, and its members are abstract methods. That has a consequence the next section exploits.

Inspector lists annotations without knowing any

Because members are methods, a tool can read an annotation it has never heard of. Stage 03's `Inspector` gets one new helper:

```
private static void appendAnnotations(StringBuilder xml, AnnotatedElement element,
String indent) {
    for (Annotation a : element.getAnnotations()) {
        Class<? extends Annotation> type = a.annotationType();
        Method[] members = type.getDeclaredMethods();
        Arrays.sort(members, Comparator.comparing(Method::getName));
        xml.append(indent).append("<annotation
type=\"").append(type.getName()).append("\"");
        if (members.length == 0) { xml.append("/>\n"); continue; }
        xml.append(">\n");
        for (Method member : members) {
            Object value = member.invoke(a);          // read the member
reflectively
            xml.append(indent).append("  <member name=\"").append(member.getName())
               .append("\">").append(escape(value)).append("</member>\n");
        }
        xml.append(indent).append("</annotation>\n");
    }
}
```

`Inspector` imports neither `Comment` nor `Range` nor `NotNull`. It asks the element for its annotations, asks each annotation for its type, asks the type for its methods, and *invokes* those methods on the annotation instance with the `Method.invoke` of stage 02. The output for `Candidate` now contains, for example:

```
  <field name="x" type="int" modifiers="private">
    <annotation type="sa4e.reflect.Range">
      <member name="max">100</member>
      <member name="min">-100</member>
    </annotation>
  </field>
  <method name="f" modifiers="public" returns="void">
    <parameter name="y" type="int">
      <annotation type="sa4e.reflect.Range">
        <member name="max">10</member>
        <member name="min">-10</member>
      </annotation>
    </parameter>
    <parameter name="d" type="double"/>
  </method>
```

This is also how an IDE, a documentation generator or a generic configuration dumper shows you annotations from libraries it has never seen.

Retention: who can see the annotation?

`RetentionPolicy` has three values, and most people have only ever consciously used one. Stage 03 defines three annotations that differ in nothing else, puts all three on a probe class, and asks who can see them:

```
@Retention(RetentionPolicy.SOURCE)  public @interface SourceOnly {}
@Retention(RetentionPolicy.CLASS)   public @interface ClassOnly {}     // the
default
@Retention(RetentionPolicy.RUNTIME) public @interface RuntimeOnly {}

@SourceOnly @ClassOnly @RuntimeOnly
public static final class Probe {}
```

Expression	Result
`Retentions.Probe.class.getAnnotations()`	`[@sa4e.reflect.Retentions.RuntimeOnly()]`
`class file attribute RuntimeVisibleAnnotations`	`[Retentions$RuntimeOnly]`
`class file attribute RuntimeInvisibleAnnotations`	`[Retentions$ClassOnly]`

Reflection sees only `RuntimeOnly`. To find out what happened to the other two we do not take anybody's word for it; we read the bytes of `Probe.class` with the ClassFile API, which Java 24 added to the JDK and which stage 07 uses in earnest:

```
byte[] bytes =
Retentions.Probe.class.getResourceAsStream("Retentions$Probe.class").readAllBytes();
ClassModel model = ClassFile.of().parse(bytes);
List<String> invisible =
model.findAttribute(Attributes.runtimeInvisibleAnnotations())
        .map(a -> a.annotations().stream().map(x ->
x.classSymbol().displayName()).toList())
        .orElse(List.of());
```

`ClassOnly` is in the class file, in an attribute literally called `RuntimeInvisibleAnnotations`. `SourceOnly` is nowhere: `javac` saw it, could have acted on it, and threw it away. So:

- `SOURCE` is for the compiler and for annotation processors: `@Override`, `@SuppressWarnings`, Lombok's `@Getter`.
- `CLASS`, the *default* when `@Retention` is omitted, is for tools that read class files without running them: bytecode analysers, obfuscators, some build-time instrumentation. If you forget `@Retention` on your own annotation and then wonder why `getAnnotation` returns `null`, this is why.
- `RUNTIME` is for frameworks. Everything in this book is `RUNTIME`.

Repeatable and inherited

Two refinements that frameworks use and that surprise people when they first meet them.

```java
@Repeatable(Tag.List.class)
@Retention(RetentionPolicy.RUNTIME)
public @interface Tag {
    String value();

    @Retention(RetentionPolicy.RUNTIME)
    @interface List { Tag[] value(); }
}

@Inherited
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Marker {}
```

```java
@Tag("alpha") @Tag("beta")
static final class Tagged extends Candidate { /* ... */ }
```

Expression	Result
`Tagged.class.getAnnotation(Tag.class)`	`null`
`Tagged.class.getAnnotation(Tag.List.class)`	`@sa4e.reflect.Tag.List({@sa4e.reflect.Tag("alpha"), @sa4e.reflect.Tag("beta")})`
`Tagged.class.getAnnotationsByType(Tag.class)`	`[@sa4e.reflect.Tag("alpha"), @sa4e.reflect.Tag("beta")]`
`Tagged.class.getAnnotation(Marker.class)`	`@sa4e.reflect.Marker() (inherited: @Marker is @Inherited)`
`Tagged.class.getAnnotation(Comment.class)`	`null (not inherited: @Comment is not)`
`Tagged.class.getDeclaredAnnotation(Marker.class)`	`null (declared on Tagged itself: no)`

Writing `@Tag` twice is syntactic sugar. The compiler wraps the repeats into one container annotation, `@Tag.List`, which is why `getAnnotation(Tag.class)` finds nothing and `getAnnotation(Tag.List.class)` finds both. `getAnnotationsByType` unwraps the container for you; use it whenever an annotation is repeatable.

`@Inherited` makes a *class* annotation visible on subclasses: `Tagged` never says `@Marker`, but `getAnnotation(Marker.class)` finds the one on `Candidate`. `@Comment` is not `@Inherited`, so it stays with `Candidate`. Two fine points: `@Inherited` works only for class-level annotations, never for methods or fields, and `getDeclaredAnnotation` ignores inheritance altogether and reports only what is written on the class itself.

Annotations are interfaces; instances are proxies

One more look at `onClass`, the `@Comment` read from `Candidate`:

Expression	Result
`onClass.getClass().getName()`	`jdk.proxy2.$Proxy1`
`Proxy.isProxyClass(onClass.getClass())`	`true`
`onClass.annotationType()`	`interface sa4e.reflect.Comment`
`onClass instanceof Annotation`	`true`
`onClass.equals(getX.getAnnotation(Comment.class))`	`false`

Nobody wrote a class that implements `Comment`. Yet `onClass` is an object, and `onClass.value()` returns a string. The object's class is called `$Proxy1`, and it was generated by the JVM when the annotation was first read, using `java.lang.reflect.Proxy`. That mechanism is the subject of stage 04; for now, note that annotations are not magic either. Also note the last line: two annotations are `equals` when their members are equal, and these two have different `value`s.

Framework one: declarative validation

Now the payoff. `@NotNull` and `@Range` are annotations with `RUNTIME` retention and targets `FIELD` and `PARAMETER`. `Validator` is a class that knows nothing about `Candidate`, or about any other class it will ever check:

```
public static List<String> validate(Object target) throws IllegalAccessException {
    List<String> violations = new ArrayList<>();
    for (Class<?> k = target.getClass(); k != null; k = k.getSuperclass()) {
        for (Field f : k.getDeclaredFields()) {
            if (Modifier.isStatic(f.getModifiers())) continue;
            f.setAccessible(true);
            Object value = f.get(target);

            if (f.isAnnotationPresent(NotNull.class) && value == null) {
                violations.add(f.getName() + " must not be null");
            }
            Range range = f.getAnnotation(Range.class);
            if (range != null && value instanceof Number number) {
                long v = number.longValue();
                if (v < range.min() || v > range.max()) {
                    violations.add(f.getName() + " = " + v
                            + " is outside [" + range.min() + ", " + range.max() +
"]");
                }
            }
        }
    }
    return violations;
}
```

stage03-annotations/src/main/java/sa4e/reflect/Validator.java

Every line is something from an earlier stage: the superclass walk, `getDeclaredFields`, `setAccessible`, `Field.get`, and now `getAnnotation`. Applied to a healthy and an unhealthy `Candidate`:

Expression	Result
`Validator.validate(new Candidate(7))`	`[]`
`Validator.validate(candidate with x=1000, label=null)`	`[x = 1000 is outside [-100, 100], label must not be null]`

(The unhealthy one had `label` set to `null` by reflection, because no constructor of `Candidate` allows it. Reflection breaks invariants as easily as it checks them.)

The point is where the knowledge lives. The rule "x must be between −100 and 100" sits on the field `x`, in `Candidate.java`, next to the thing it constrains. The code that enforces it is generic and

shared. This is Jakarta Bean Validation in thirty lines; the real thing has hundreds of constraints, message interpolation, groups and cascading, but the shape is identical. Every web framework's form binding is this loop.

Framework two: a test runner

The second framework introduces the idea the rest of the book is built on. Three annotations:

```
@Retention(RUNTIME) @Target(METHOD) public @interface Test {}
@Retention(RUNTIME) @Target(METHOD) public @interface BeforeEach {}
@Retention(RUNTIME) @Target(METHOD) public @interface Disabled { String value()
default ""; }
```

a test class that uses them and never calls its own methods:

```
public class CandidateTests {
    private Candidate c;

    @BeforeEach
    void setUp() { c = new Candidate(7); }

    @Test
    void negativeDKeepsY() { c.f(3, -1.0); assertEquals(3, c.getX()); }

    @Test
    void positiveDNegatesY() { c.f(3, 1.0); assertEquals(-3, c.getX()); }

    @Test
    void resetSetsFortyTwo() { c.reset(); assertEquals(42, c.getX()); }

    @Test
    void deliberateFailure() { assertEquals(8, c.getX()); }   // wrong on purpose

    @Test @Disabled("waiting for stage 04")
    void notYet() {}

    private static void assertEquals(int expected, int actual) {
        if (expected != actual) throw new AssertionError("expected " + expected + "
but was " + actual);
    }
}
```

and a runner:

```
public static List<Result> run(Class<?> testClass) throws
ReflectiveOperationException {
    Method[] methods = testClass.getDeclaredMethods();
    Arrays.sort(methods, Comparator.comparing(Method::getName));   // reflection
promises no order
    List<Method> setups = new ArrayList<>();
    List<Method> tests = new ArrayList<>();
    for (Method m : methods) {
        m.setAccessible(true);                                     // tests are
usually package-private
        if (m.isAnnotationPresent(BeforeEach.class)) setups.add(m);
        if (m.isAnnotationPresent(Test.class)) tests.add(m);
```

```
    }

    List<Result> results = new ArrayList<>();
    for (Method test : tests) {
        Disabled disabled = test.getAnnotation(Disabled.class);
        if (disabled != null) {
            results.add(new Result(test.getName(), "skipped", disabled.value()));
            continue;
        }
        Object instance = testClass.getDeclaredConstructor().newInstance();   //
fresh per test
        try {
            for (Method setup : setups) setup.invoke(instance);
            test.invoke(instance);
            results.add(new Result(test.getName(), "passed", ""));
        } catch (InvocationTargetException e) {                    // the test
threw: unwrap
            Throwable cause = e.getCause();
            results.add(new Result(test.getName(), "failed",
                    cause.getClass().getSimpleName() + ": " + cause.getMessage()));
        }
    }
    return results;
}
```

stage03-annotations/src/main/java/sa4e/reflect/TestRunner.java

Test	Outcome	Detail
`deliberateFailure`	failed	AssertionError: expected 8 but was 7
`negativeDKeepsY`	passed	
`notYet`	skipped	waiting for stage 04
`positiveDNegatesY`	passed	
`resetSetsFortyTwo`	passed	

Every mechanical detail is familiar by now: find methods, read annotations, create an instance, invoke, unwrap `InvocationTargetException`. What is new is the *direction of control*. `CandidateTests` is a class with five methods that nobody in `CandidateTests` ever calls. The runner decides what to create, in what order to call what, and what a failure means. The test class only *declares*, through annotations, which methods matter.

That reversal, where the framework calls your code instead of your code calling the framework, has a name: *inversion of control*, sometimes summarised as the Hollywood principle, “don’t call us, we’ll call you”. JUnit is this runner with lifecycle extensions, parameter resolution and a reporting API. Servlet containers, message listeners, GUI event loops and, in stage 06, dependency-injection containers all work this way. Stage 03 is where the book’s title quietly changes from “reflection” to “and more”.

In the wild

- Jakarta Bean Validation (`@NotNull`, `@Min`, `@Max`, `@Size`) is the validator with hundreds of constraints and message interpolation.

- JUnit 5 is the test runner with extension points; `@BeforeEach`, `@Test` and `@Disabled` are its real names.
- JPA (`@Entity`, `@Column`), Jackson (`@JsonProperty`), Spring (`@Component`, `@Transactional`) and Jakarta REST (`@GET`, `@Path`) are all "framework reads annotation, framework acts". Stages 04 and 06 build two of them.
- Annotations with `CLASS` or `SOURCE` retention feed annotation processors at compile time (Lombok, MapStruct, Dagger), which generate code instead of reflecting at run time.

The bill

- Annotations are declarations, not behaviour. Nothing happens unless some code reads them; a forgotten framework call silently ignores every constraint, and the compiler cannot tell you.
- The same annotation can mean different things to different frameworks, and the order in which frameworks process them is expressed nowhere.
- Reading annotations is reflection, with stage 02's costs. Frameworks scan once at startup and cache.
- Forgetting `@Retention(RUNTIME)` is the single most common annotation bug, and its symptom is a silent `null`.

Try it yourself

1. Add `@Size(min, max)` for strings and collections to `Validator`.
2. Let `TestRunner` support `@AfterEach`, and run only the tests whose names are given on the command line.
3. Give `@Comment` a second member `author` with a default, and find out whether an annotation instance can tell which of its members were defaulted. (It cannot. Where could the information come from?)
4. Declare an annotation with `SOURCE` retention and verify with `javap -v` that it is absent from the class file; then do the same for `CLASS` retention and find it in `RuntimeInvisibleAnnotations`.

Takeaways

- An annotation is an interface with `@Target` and `@Retention`; its members are parameterless methods with optional defaults.
- Only `RUNTIME` retention is visible to reflection. `CLASS` is the default and is stored but invisible; `SOURCE` never leaves `javac`.
- `getAnnotation`, `isAnnotationPresent` and `getAnnotationsByType` work on every `AnnotatedElement`; annotation instances are generated proxies.
- A framework is a loop over reflection plus annotations: find the annotated elements, then act. The validator and the test runner are that loop.
- When the framework calls your code, that is inversion of control.